



CS-317 OPERATING SYSTEMS

Lecture 4



PROCESSES

Book 1 Chapter 3

Things we learned in the last lecture...

- In the last lecture, we understood the process image.
- We talked about the Process Control Block in detail.
- We understood the data structures the OS uses to manage processes.
- We realized the role of the PCB in an OS.



PROCESS CONTROL

Mechanisms of the OS for process management

Modes of execution

- The processor has at least two modes of execution: the **kernel** mode, alternatively called **system** mode or **control** mode, and the **user** mode.
- **Privileged instructions** are instructions that can only be executed in the kernel mode. All other instructions are called **user instructions**. Privileged instructions include those that read or alter a control register like the PSW, primitive I/O instructions, and instructions relevant to memory management.
- OS executes in the kernel mode. All applications run in the user mode.
- OS is the only entity that can execute privileged instructions.

Table 3.7 Typical Functions of an Operating System Kernel

Process Management
• Process creation and termination • Process scheduling and dispatching • Process switching • Process synchronization and support for interprocess communication • Management of process control blocks
Memory Management
• Allocation of address space to processes • Swapping • Page and segment management
I/O Management
• Buffer management • Allocation of I/O channels and devices to processes
Support Functions
• Interrupt handling • Accounting • Monitoring

Functions of the OS' kernel

The **kernel** of the OS is the portion of the OS encompassing the most important system functions.

How is the processor's mode determined?

- There is a bit in the Program Status Word (PSW) which when reset, means the processor is working in one mode, and when set, means the processor is working in the other mode.
- Changing this bit clearly means changing the mode of the processor.
- A change in the mode of the processor, from user to kernel, or from kernel to user, is called a **mode switch**.
- A mode switch happens in response to certain events.

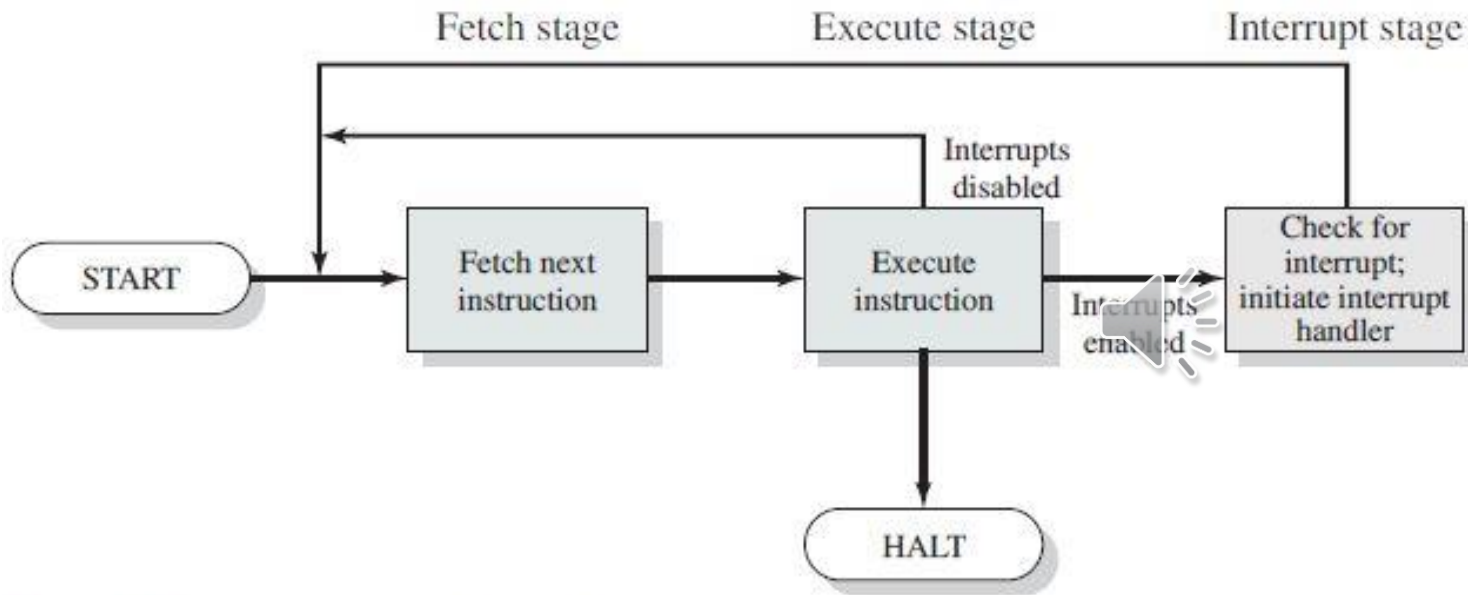


Figure 1.7 Instruction Cycle with Interrupts

Mode switching in interrupt handling

If an interrupt signal is pending,

1. The processor loads the address of an interrupt handler routine in the program counter.
2. It switches from user mode to kernel mode.

Steps of process creation



Assign a unique identifier to the new process.



Allocate space for the process.



Initialize the PCB.



Set the appropriate linkages.



Create or expand other data structures.

Process switching

- When control is transferred from a user process to another user process due to some reason, this is called a **process switch** or a **context switch**.
- A process switch comprises:
 - Transfer of control from the running process to the OS,
 - Transfer of control from the OS to another process.
- A process switch may occur any time the OS regains control of the processor from the running process.



Table 3.8 Mechanisms for Interrupting the Execution of a Process

Mechanism	Cause	Use
Interrupt	External to the execution of the current instruction	Reaction to an asynchronous external event 
Trap	Associated with the execution of the current instruction	Handling of an error or an exception condition
Supervisor call	Explicit request	Call to an operating system function

Mechanisms that interrupt process execution

Three mechanisms interrupt a running process:

- Interrupt
- Trap
- Supervisor call



System interrupts

System interrupts are of two types:

- The ordinary **interrupt**, and
- The **trap**.

Interrupt

- It is generated due to an event that is external to and independent of the currently running process.
- There are three kinds of the ordinary interrupt:
 - Clock interrupt,
 - I/O interrupt,
 - Memory fault.
- When an ordinary interrupt occurs:
 1. The processor is taken away from the running process.
 2. A generic interrupt handler routine starts to execute.
 3. The generic interrupt handler transfers control to an OS routine that is concerned with this interrupt.
 4. The interrupt specific routine completes and passes control to the OS.
 5. The OS may pass control to the interrupted process or to a different process.



Clock interrupt

- This interrupt comes from the clock indicating that the time slot of the running process has expired.
- After the clock interrupt, control passes from the running process to the interrupt handler.
- The interrupt handler passes control on to the dispatcher.
- The dispatcher selects a process from the ready queue.
- The selected process starts execution on the processor.
- Every clock interrupt leads to a process switch.

I/O interrupt

- This interrupt comes from an I/O device indicating that the I/O action it was performing is complete.
- After the I/O interrupt, control passes from the running process to the interrupt handler.
- The interrupt handler passes the control on to OS.
- The OS identifies processes which are waiting for this event.
- The OS moves the waiting processes into appropriate queues.
- The OS determines if a process, higher in priority than the interrupted process, has been added to the ready or ready/suspend queues. If this is the case, process switch occurs. Otherwise, control is passed back to the interrupted process.
- An I/O interrupt may or may not lead to a process switch.

Task# 1: Can you think of a scenario where a clock interrupt does not lead to a process switch?

no other process in the queue



Task# 2: Can an I/O interrupt be related in any way to the running process?

An I/O interrupt is ****usually not related to the currently running process****, but to a process that previously requested I/O and is now blocked.

Pause the video!

Write your answer in a word file. Save it as <my_roll_no>_lecture_4. If your roll no is 500, the file will be named 500_lecture_4. Do not add <> signs to the filename.

Memory fault

- This interrupt indicates that the address for which the processor has generated a memory reference is not loaded in main memory. It is also called a **page fault**.
- After the memory fault, control passes from the running process to the memory management part of the OS, which issues an I/O request to bring the page or segment containing the desired address into main memory.
- The interrupted process is moved to the blocked queue.
- The OS passes control on to the dispatcher.
- The dispatcher selects a process from the ready queue.
- The selected process starts execution on the processor.
- Every memory fault leads to a process switch.

- The memory fault is generated when the processor asks for a piece of data not loaded in main memory.
- There is no bug or error in the code of the running process.
- This happens because we partially load processes in main memory, as we do not have enough space in the RAM. So it is the fault of the memory management system, not the running process.
- If the running process is run in a different virtual memory environment, or in the same environment with a different number of processes, the memory fault may not occur. Or it may happen that a memory fault for a different word occurs. This shows that memory faults occur due to the system state and not because of the running process.

How is a memory fault external to and independent of the currently running process?

Trap

- A trap is a system interrupt that occurs because of an error or an exception condition generated within the currently running process.
- If the error or exception condition is fatal, the interrupted process is moved to the exit state. The OS transfers control to the dispatcher. The dispatcher selects a process from the ready queue. The selected process starts execution on the processor. A process switch has just occurred.
- If the error or exception condition is not fatal, the succeeding actions will vary according to OS and error. The OS may attempt recovery, or it may simply notify the user. If the OS can recover from the error, or the user helps to recover from it, the OS may resume the running process. If recovery is not possible or if it is taking time, the OS might do a process switch.
- A trap may or may not lead to a process switch.

Supervisor Call

- The running process calls a function of the OS. This is called a **supervisor call** or an **operating system service call**.
- The running process relinquishes the processor and control passes to the OS.
- The OS directs the relevant module to complete the request made by the process.
- If the process' request will take time to complete, the process is placed in the blocked queue. The OS transfers control to the dispatcher. The dispatcher selects a process from the ready queue. The selected process starts execution on the processor. A process switch has just occurred.
- If the process' request can be readily satisfied, the OS resumes the process who made the supervisor call.
- Most supervisor calls result in process switching.

Task# 3: A memory fault occurs. The address space of the running process has no room for the incoming page. What will you do?

When no free frame is available, the OS performs ****page replacement****: it selects a victim page (using an algorithm like LRU or FIFO), writes it to disk if modified, frees the frame, loads the required page into memory, updates the page table, and resumes execution.



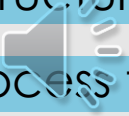
Task# 4: When I click *New*, then *Blank Document*, in Microsoft Word, the running process makes a supervisor call. What is it asking the OS to do?

It is asking the OS to create a new empty file (document), allocate resources, and prepare it for editing.

Pause the video!

Write your answer in a word file previously saved as <my_roll_no>_lecture_4.

Steps of process switch

1. Save the context of the processor, including program counter and other registers.
2. Update the PCB of the running process. This includes changing its state from running to an appropriate state. The reason for leaving execution and accounting information will also be recorded in the PCB.
3. Move the PCB to the appropriate queue. 
4. Select another process for execution.
5. Update the PCB of the selected process. This includes changing its state from ready to running.
6. Update memory management data structures. 
7. Restore the context of the selected process to that which existed in when it was switched out last time. This means loading the values of the program counter and registers that were stored in its PCB when it was switched out.

Mode switching for process switch

Start of a process switch

1. If an interrupt signal is pending, a trap has occurred, or a supervisor call is made, the program counter is adjusted to the address of an OS function or module.
2. The processor switches from user mode to kernel mode.
3. The context of the running process is saved from the processor's registers into its PCB.

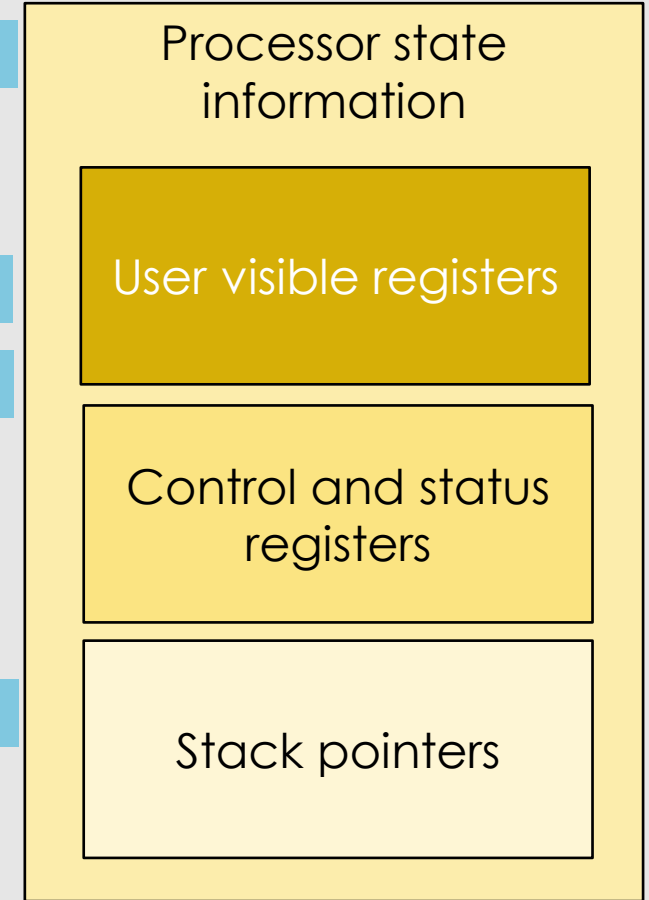
Mode switching for process switch

End of a process switch

1. The context of the selected process is restored from its PCB into the processor's registers.
2. The processor switches from kernel mode to user mode.
3. The program counter is adjusted to the address of the instruction of the selected program to be executed next.

What is the context of a process?

- The portion of the PCB referred to as processor state information constitutes the context of the process.
- Processor state information includes program counter, processor's registers and stack information.
- The context of the process is **saved** in the PCB when switched out.
- The context of the process is **restored** on the processor when switched in.
- Saving and restoring the context allows a process to pick up its execution from the point at which it was interrupted.



Difference between process and mode switch

- A process switch is the changing of user processes on the processor.
- A mode switch is the changing of execution mode of the processor.
- Every process switch incorporates two mode switches.
- A mode switch may or may not be a part of events that lead to process switch.

Concluding remarks

- In this lecture we understood how processes are controlled.
- We talked about the process switch, and the events that may lead to it.
- We understood the mode switch.
- We tied the two switches together and saw how they happen in an OS.



Things to do

- Read the book.
- I will ask you to submit the word file previously saved as <my_roll_no>_lecture_4 in  Google classroom.
- Write your questions on a piece of paper and keep it in front of you during the online session.
- If you have suggestions relevant to content or quality of this lecture, email them to me.